

BÁO CÁO KẾT QUẢ THỬ NGHIỆM

Sinh viên thực hiện: Đặng Xuân Bách; MSSV: 25520109.

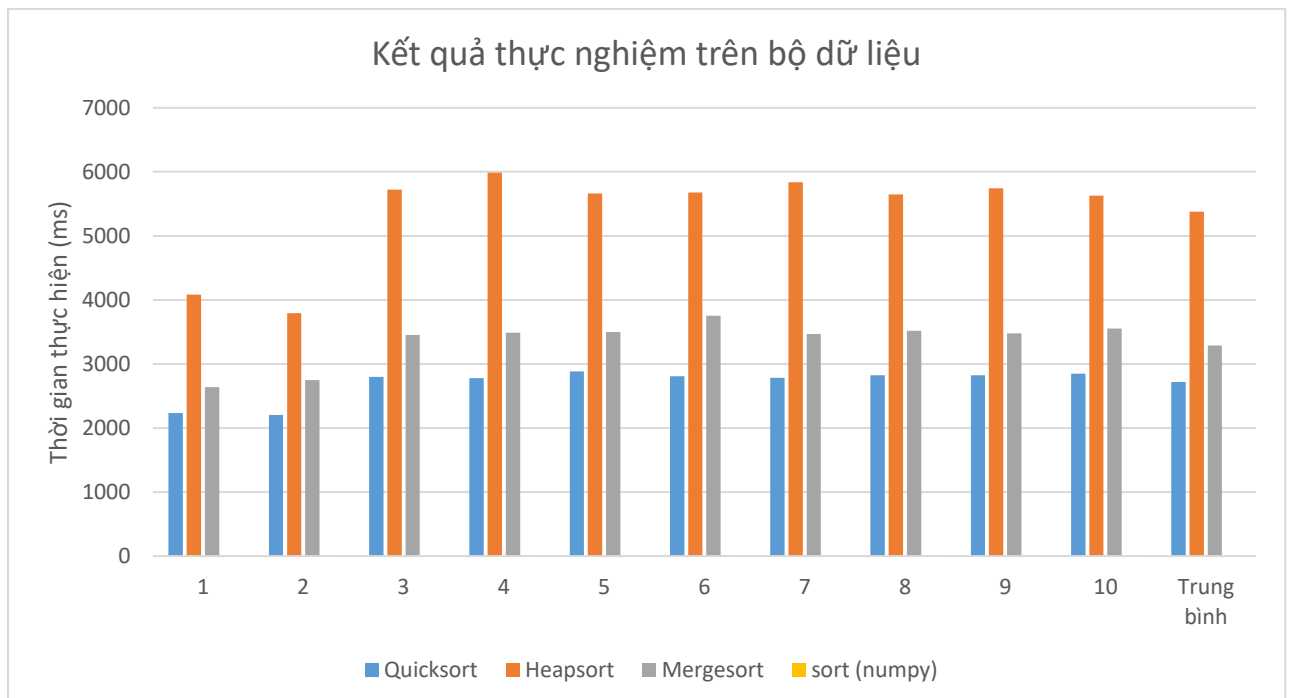
Nội dung báo cáo:

I. Kết quả thử nghiệm

1. Bảng thời gian thực hiện¹

Dữ Liệu	Quicksort	Heapsort	Mergesort	sort (numpy)
1	2234.58	4085.22	2638.52	10.10
2	2202.91	3794.57	2750.38	7.67
3	2800.62	5723.26	3452.38	7.96
4	2777.25	5985.48	3490.42	7.80
5	2885.80	5663.50	3499.21	7.51
6	2807.11	5679.86	3753.70	7.37
7	2782.37	5835.35	3466.44	9.59
8	2823.64	5645.99	3517.68	8.95
9	2823.64	5740.53	3476.10	8.08
10	2850.05	5628.16	3550.73	7.96
Trung bình	2718.60	5378.65	3289.12	8.30

2. Biểu đồ (cột) thời gian thực hiện



II. Kết luận:

¹ Số liệu chỉ mang tính minh họa

- **Về hiệu năng:** Qua bảng số liệu thực nghiệm trên 10 bộ dữ liệu (mỗi bộ 1 triệu phần tử), ta thấy sự chênh lệch rõ rệt giữa các nhóm thuật toán:
- **Nhóm thư viện (Numpy):** Đạt hiệu năng vượt trội (Numpy ~8.3ms). Điều này là dễ hiểu do các thư viện này được tối ưu hóa sâu ở mức ngôn ngữ máy (C/Assembly) và quản lý bộ nhớ tốt.
- **Nhóm tự cài đặt (Quick, Heap, Merge):** Có thời gian thực thi cao hơn, dao động từ 2700 ms đến 5400 ms. Trong đó, Quicksort là thuật toán nhanh nhất.

Cả ba thuật toán dù đều có độ phức tạp $O(N \log N)$ nhưng mỗi thuật lại có ưu và nhược điểm riêng về mặt thực tế:

- **Quick Sort (2718.60 ms):** Chiếm ưu thế nhất nhóm tự cài đặt nhờ khả năng tận dụng bộ nhớ đệm (Cache locality) tốt và hệ số hằng số nhỏ. Nhưng lại chậm hơn trong một số trường hợp. Trong đoạn thử nghiệm em đã dùng thuật random pivot để tránh các trường hợp này.
- **Merge Sort (3289.12 ms):** Chậm hơn Quicksort một chút nhưng vẫn giữ hiệu năng ổn định. Điểm yếu là tốn thêm bộ nhớ phụ, nhưng ưu điểm lớn nhất là tính ổn định (Stable sort), bảo toàn thứ tự các phần tử bằng nhau.
- **Heap Sort (5378.65 ms):** Trong bài thử nghiệm này, Heapsort cho kết quả chậm nhất. Dù có ưu điểm là độ phức tạp luôn đảm bảo $O(n \log n)$ và không tốn bộ nhớ phụ như Mergesort, nhưng việc truy cập bộ nhớ không liên tục khi duy trì cấu trúc Heap khiến nó kém hiệu quả hơn về mặt thực tế trên tập dữ liệu lớn này.

III. Thông tin chi tiết – link github, trong repo gibub cần có

1. Mã nguồn: [EvolutionzXD/IT003.Q21.CTTN-PROJECT1: Thực nghiệm các giải thuật sắp xếp nội](#)
2. Dữ liệu thử nghiệm: [EvolutionzXD/IT003.Q21.CTTN-PROJECT1: Thực nghiệm các giải thuật sắp xếp nội](#)